Yes provide me the comprehensive instructions, all the structure etc. in one go

Use Python with Scrapy as the default crawler, JSONL as the raw export, Markdown as the readable report, and `launchd` for the daily scheduler on macOS. That stack fits your terminal-first workflow, because Scrapy is built for crawling and extraction, Scrapy feed exports are configured through `FEEDS`, and Apple documents `launchd` with `StartCalendarInterval` for timed jobs on macOS.[1] [2] [3]

Stack

This is the recommended stack for version 1 of your project.[2] [4] [3]

Part	Use
Language	Python 3, because you want a terminal-run scraper and one-language workflow. [3]
Crawler	Scrapy first, because it is designed for crawling websites and extracting structured data. [3]
Optional browser	Playwright for Python only when a page requires JavaScript rendering or browser automation. [5]
Raw storage	JSONL via Scrapy FEEDS, because feed exports are built in and configurable. [1]
Human report	Plain Python script that reads JSONL and writes `.md`.
Scheduler	`launchd`, because Apple documents it as the timed-job mechanism on macOS via `StartCalendarInterval`. [2]
Crawl safety	`ROBOTSTXT_OBEY`, `DOWNLOAD_DELAY`, low concurrency, retries, and logging. [2] [4] [4]

For OnlineJobs.ph, keep the crawler slow, because its `robots.txt` specifies `Crawl-delay: 5` and disallows some paths. For Upwork, keep it out of the automated daily run until you confirm an approved API or allowed automation path, because recent guidance separates approved API use from restricted bot behavior.[4] [6]

Structure

Use this folder layout so raw data, reports, logs, and code stay separate and easy to maintain.

```
job-crawler/
├── .venv/
├── scrapy.cfg
├── requirements.txt
├── run_daily.sh
├── jobscan.plist
```

```
├── config/
│   └── keywords.yaml
├── data/
│   ├── raw/
│   └── snapshots/
├── reports/
├── logs/
└── crawler/
    ├── __init__.py
    ├── items.py
    ├── settings.py
    ├── pipelines.py
    ├── utils/
    │   ├── markdown_report.py
    │   └── dedupe.py
    └── spiders/
        ├── onlinejobs.py
        └── upwork.py
```

Keep `upwork.py` as a placeholder adapter only, and do not wire it into the scheduled run until you confirm a compliant access method. Keep `onlinejobs.py` as the first live spider, with robots obeyed and delay enforced from day one.[6] [2] [4]

Install

Create a virtual environment, install Scrapy first, and only add Playwright later if normal HTML crawling is not enough.[3] [5]

```
mkdir -p ~/job-crawler
cd ~/job-crawler

python3 -m venv .venv
source .venv/bin/activate

pip install --upgrade pip
pip install scrapy pyyaml python-dateutil
# optional later only if needed:
# pip install playwright
# playwright install
```

Add a simple `requirements.txt`:

```
scrapy
pyyaml
python-dateutil
# optional later:
# playwright
```

Scrapy's feed export system is configured through the `FEEDS` setting, so let Scrapy write JSONL and let your own Python script handle Markdown generation after the crawl finishes. That split

is cleaner than trying to force the crawler itself to write presentation-ready Markdown directly.[1]

Files

Your first working build only needs five real files: `settings.py`, `items.py`, `spiders/onlinejobs.py`, `utils/markdown_report.py`, and `run_daily.sh`.[4] [3] [1]

`crawler/items.py`

```
import scrapy

class JobItem(scrapy.Item):
    source = scrapy.Field()
    source_id = scrapy.Field()
    title = scrapy.Field()
    company = scrapy.Field()
    pay = scrapy.Field()
    posted_at = scrapy.Field()
    url = scrapy.Field()
    summary = scrapy.Field()
    scraped_at = scrapy.Field()
    is_new = scrapy.Field()
```

`crawler/settings.py`

```
BOT_NAME = "jobcrawler"

SPIDER_MODULES = ["crawler.spiders"]
NEWSPIDER_MODULE = "crawler.spiders"

ROBOTSTXT_OBEY = True
DOWNLOAD_DELAY = 5
CONCURRENT_REQUESTS_PER_DOMAIN = 1

AUTOTHROTTLE_ENABLED = True
AUTOTHROTTLE_START_DELAY = 5
AUTOTHROTTLE_MAX_DELAY = 30

RETRY_ENABLED = True
RETRY_TIMES = 2

FEEDS = {
    "data/raw/jobs-%(time)s.jsonl": {
        "format": "jsonlines",
        "encoding": "utf8",
        "overwrite": False
    }
}

LOG_FILE = "logs/scrapy.log"
LOG_LEVEL = "INFO"
USER_AGENT = "jobcrawler/1.0 (+personal research crawler)"
```

Those settings matter because Scrapy supports feed exports through `FEEDS`, while `ROBOTSTXT_OBEY` and a 5-second delay match the root robots guidance and OnlineJobs.ph's published crawl delay.[1] [4]

`config/keywords.yaml`

```
keywords:
  - seo specialist
  - link building
  - outreach
  - content writer
```

`crawler/spiders/onlinejobs.py`

```
import scrapy
from datetime import datetime, timezone
from urllib.parse import quote
from crawler.items import JobItem

class OnlineJobsSpider(scrapy.Spider):
    name = "onlinejobs"
    allowed_domains = ["onlinejobs.ph"]

    def __init__(self, keyword=None, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.keyword = keyword or "seo specialist"

    def start_requests(self):
        q = quote(self.keyword)
        url = f"https://www.onlinejobs.ph/jobseekers/jobsearch?jobkeyword={q}"
        yield scrapy.Request(url, callback=self.parse_search)

    def parse_search(self, response):
        job_links = response.css('a::attr(href)').getall()
        for href in job_links:
            if "/jobseekers/job/" in href:
                yield response.follow(href, callback=self.parse_job)

        next_page = response.css('a[rel="next"]::attr(href)').get()
        if next_page:
            yield response.follow(next_page, callback=self.parse_search)

    def parse_job(self, response):
        title = response.css("h1::text").get(default="").strip()
        body_text = " ".join(t.strip() for t in response.css("body *::text").getall() if

        item = JobItem()
        item["source"] = "onlinejobs"
        item["source_id"] = response.url.rstrip("/").split("-")[-1]
        item["title"] = title
        item["company"] = ""
        item["pay"] = ""
        item["posted_at"] = ""
        item["url"] = response.url
```

```
        item["summary"] = body_text[:500]
        item["scraped_at"] = datetime.now(timezone.utc).isoformat()
        item["is_new"] = True
        yield item
```

`crawler/utils/markdown_report.py`

```
import json
from pathlib import Path
from datetime import datetime

raw_dir = Path("data/raw")
report_dir = Path("reports")
report_dir.mkdir(parents=True, exist_ok=True)

latest = sorted(raw_dir.glob("jobs-*.jsonl"))[-1]
rows = [json.loads(line) for line in latest.read_text(encoding="utf-8").splitlines() if l

today = datetime.now().strftime("%Y-%m-%d")
new_count = sum(1 for r in rows if r.get("is_new"))

lines = [
    f"# Daily Job Scan - {today}",
    "",
    f"- Total jobs found: {len(rows)}",
    f"- New jobs found: {new_count}",
    "",
    "## Jobs",
    ""
]

for r in rows:
    lines.extend([
        f"### {r.get('title','No title')}",
        f"- Source: {r.get('source','')}",
        f"- URL: {r.get('url','')}",
        f"- Summary: {r.get('summary','')}",
        ""
    ])

out = report_dir / f"jobs-{today}.md"
out.write_text("\n".join(lines), encoding="utf-8")
```

`run_daily.sh`

```
#!/bin/zsh
set -e

cd "$HOME/job-crawler"
source .venv/bin/activate

mkdir -p data/raw reports logs
```

```
scrapy crawl onlinejobs -a keyword="seo specialist"
python -m crawler.utils.markdown_report >> logs/report.log 2>&1
```

Make it executable:

```
chmod +x ~/job-crawler/run_daily.sh
```

Automation

Use a per-user `launchd` plist with `StartCalendarInterval` to run the shell script once a day, because Apple documents that timed jobs on macOS use property lists and `StartCalendarInterval`. A secondary source summarizing Apple's behavior notes that `launchd` jobs scheduled this way run on wake if the Mac was asleep at the scheduled time, unlike cron jobs that are skipped while the computer sleeps.[7] [2]

`jobscan.plist`

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/Property
<plist version="1.0">
  <dict>
    <key>Label</key>
    <string>com.genyl.jobscan</string>

    <key>ProgramArguments</key>
    <array>
      <string>/bin/zsh</string>
      <string>/Users/YOUR_USERNAME/job-crawler/run_daily.sh</string>
    </array>

    <key>StartCalendarInterval</key>
    <dict>
      <key>Hour</key>
      <integer>7</integer>
      <key>Minute</key>
      <integer>0</integer>
    </dict>

    <key>RunAtLoad</key>
    <false/>

    <key>StandardOutPath</key>
    <string>/Users/YOUR_USERNAME/job-crawler/logs/launchd.out.log</string>

    <key>StandardErrorPath</key>
    <string>/Users/YOUR_USERNAME/job-crawler/logs/launchd.err.log</string>
  </dict>
</plist>
```

Load it with:

```
launchctl unload ~/Library/LaunchAgents/com.genyl.jobscan.plist 2>/dev/null || true
cp ~/job-crawler/jobscan.plist ~/Library/LaunchAgents/com.genyl.jobscan.plist
launchctl load ~/Library/LaunchAgents/com.genyl.jobscan.plist
launchctl list | grep com.genyl.jobscan
```

Your daily workflow should be: crawl OnlineJobs.ph once per day, export JSONL through Scrapy, convert the latest JSONL into Markdown, and review the report in `reports/`. Keep retries low, concurrency low, and the delay at 5 seconds for OnlineJobs.ph, because its published robots file explicitly includes that crawl delay. Keep the Upwork adapter disabled until you confirm an approved API or other allowed automation path, because the policy risk there is different from a normal public-site crawl.[6] [4] [1]

Do you want me to turn this into copy-paste-ready file contents for every file in the project next?

⁂

1. https://docs.scrapy.org/en/latest/topics/feed-exports.html
2. https://developers.google.com/search/docs/crawling-indexing/robots/intro
3. https://www.onlinejobs.ph/jobseekers/job/Website-Needs-improvement-and-ranking-in-1st-page-1099864
4. https://developers.google.com/crawling/docs/robots-txt/create-robots-txt
5. https://playwright.dev/python/docs/intro
6. https://gigradar.io/blog/responsible-automation-on-upwork
7. https://alvinalexander.com/mac-os-x/launchd-plist-examples-startinterval-startcalendarinterval/
8. https://docs.scrapy.org/en/latest/topics/settings.html
9. https://docs.scrapy.org/en/0.24/topics/feed-exports.html
10. https://www.geeksforgeeks.org/python/scrapy-feed-exports/
11. https://discussions.apple.com/thread/241734
12. https://docs.scrapy.org/en/latest/topics/exporters.html
13. https://stackoverflow.com/questions/37084598/launchd-runs-with-manual-launchctl-start-but-not-on-startcalendarinterval
14. https://www.reddit.com/r/buhaydigital/comments/1p3hwyb/built_an_n8n_automation_for_job_hunting_on/
15. https://scrapyd.readthedocs.io/en/latest/config.html
16. https://developer.apple.com/library/archive/documentation/MacOSX/Conceptual/BPSystemStartup/Chapters/CreatingLaunchdJobs.html
17. https://docs.scrapy.org/en/1.4/topics/feed-exports.html
18. https://docs.scrapy.org/en/1.1/topics/feed-exports.html